

Evaluación de Calidad del Software — ISO/IEC 25010

Proyecto Cámaras-IA · Duoc UC

Norma aplicada: ISO/IEC 25010:2011 — *Systems and software Quality Requirements and Evaluation (SQuaRE)*
Versión del documento: 1.0 **Fecha de evaluación:** Junio 2026 **Equipo evaluador:** Ignacio Fuenzalida
Ramírez · Nicolas Alejandro Arcos Alfaro **Ambiente evaluado:** Producción — Oracle Cloud Infrastructure
(ARM, 4 vCPU, 24 GB RAM)

1. Introducción

La norma **ISO/IEC 25010** define un modelo de calidad del producto software compuesto por **8 características principales** y sus sub-características asociadas. Este documento aplica dicho modelo al sistema **Cámaras-IA**, evaluando cada característica con evidencia extraída directamente del código fuente, la configuración de infraestructura y los resultados de las pruebas de aceptación.

El objetivo no es solo verificar que el software *funciona*, sino demostrar que cumple con **atributos de calidad** que garantizan su valor a largo plazo: que es seguro, mantenible, eficiente, confiable y portable.

2. Resumen Ejecutivo de Calidad

#	Característica ISO 25010	Nivel Alcanzado	Calificación
1	Adecuación Funcional	Sistema cumple todos los objetivos de negocio definidos	★★★★★
2	Eficiencia de Desempeño	Latencia YOLO < 2s · Restricción conocida en LLaVA CPU	★★★★
3	Compatibilidad	Integración exitosa entre todos los componentes	★★★★★
4	Usabilidad	Interfaz web responsiva · Alertas comprensibles	★★★★
5	Fiabilidad	Heartbeats · Reintentos · Modo degradado	★★★★
6	Seguridad	JWT + bcrypt + control de roles + HTTPS	★★★★★
7	Mantenibilidad	Código documentado · Arquitectura modular · Docker	★★★★★
8	Portabilidad	Contenerizado 100% · Desplegable en cualquier cloud	★★★★★

[!NOTE] **Calificación global: 4.6 / 5.0** — Sistema de alta calidad con restricciones conocidas y documentadas en eficiencia de desempeño por el uso de infraestructura gratuita.

3. Evaluación por Característica

3.1 Adecuación Funcional

¿El software hace exactamente lo que se necesita?

La Adecuación Funcional evalúa si el sistema proporciona las funciones que satisfacen las necesidades implícitas y explícitas del usuario final, con la corrección y completitud necesarias.

3.1.1 Completitud Funcional

El sistema cubre la totalidad de las funciones definidas en los requisitos del Proyecto de Título:

Requisito	Función implementada	Endpoint / Componente
Monitoreo en tiempo real	Mosaico de cámaras vía HLS	GET /camaras + MediaMTX
Detección de personas	Pipeline YOLOv8	detector.py
Análisis semántico	Descripción LLaVA de cada alerta	analizador2.py
Registro de alertas	Tabla alertas en PostgreSQL	POST interno desde detector
Notificación push	Bot Telegram con polling de 5s	Contenedor camaras-telegram
Gestión de usuarios	CRUD completo con roles	GET/POST/PUT/DELETE /usuarios
Configuración del sistema	Credenciales RTSP editables en UI	PATCH /camaras/{id}/credenciales
Historial de grabaciones	Almacenamiento en MinIO	GET /grabaciones
Monitoreo de salud	Dashboard de estado de servicios	GET /health · GET /health/ia

3.1.2 Corrección Funcional

- **Hashing de contraseñas:** Implementado con bcrypt a 12 rondas (\$2b\$12\$...). La función trunca correctamente a 72 bytes según el límite de bcrypt.
- **Tokens JWT:** Firmados con HS256, con tiempo de expiración de 1440 minutos (24 horas). Validados en cada petición protegida.
- **Descubrimiento de cámaras:** El sistema encuentra la IP de cada cámara por su dirección MAC en la tabla ARP de la red local, con fallback a la IP de respaldo almacenada en la base de datos.

3.1.3 Pertinencia Funcional

El sistema no implementa funciones innecesarias que compliquen su uso. Las funciones están organizadas en 9 grupos de endpoints con etiquetas claras: auth, usuarios, camaras, alertas, grabaciones, snapshots, eventos, realtime, meta.

Calificación: ★ ★ ★ ★ ★ (5/5)

3.2 Eficiencia de Desempeño

¿Usa bien los recursos? ¿Es rápido?

3.2.1 Comportamiento Temporal (Latencias medidas)

Operación	Latencia Medida	Objetivo	Resultado
Detección YOLO → Alerta en frontend	~1.3 segundos	< 2.0 s	✓ Cumplido
Respuesta de GET /health	< 200 ms	< 500 ms	✓ Cumplido
Análisis semántico LLaVA (CPU)	~120 segundos	< 10 s (GPU)	⚠ Limitación conocida
Worker Telegram (detección → notif)	< 10 segundos	< 30 s	✓ Cumplido
Carga del listado de alertas	< 400 ms	< 1 s	✓ Cumplido

3.2.2 Utilización de Recursos

El sistema está diseñado para optimizar los recursos disponibles:

- **Redis** actúa como cola de mensajes entre el detector local y el backend en Oracle, evitando que las tareas de IA pesada bloqueen el hilo principal de la API.
- **Cooldown de alertas configurado en 30 segundos** (`CAMARAS_ALERT_COOLDOWN_S: "30"` en `docker-compose.yml`) para evitar la saturación de la base de datos y del canal de Telegram ante detecciones continuas.
- **Pool de conexiones a PostgreSQL** gestionado por `psycopg` con `connection_pool` para reutilizar conexiones y evitar el overhead de establecer una nueva conexión por cada petición.

3.2.3 Capacidad

La infraestructura Oracle (4 vCPU ARM, 24 GB RAM) soporta cómodamente los 6 contenedores Docker del sistema sin necesidad de escalado horizontal para el volumen de cámaras actual (2 cámaras).

[!WARNING] **Limitación documentada:** La latencia de 120 segundos de LLaVA en CPU es una restricción arquitectural conocida del uso de infraestructura gratuita. La ruta de escalamiento identificada es migrar a una instancia con GPU, donde la latencia se reduciría a menos de 3 segundos.

Calificación: ★★☆☆ (4/5) — Por la restricción conocida de LLaVA en CPU.

3.3 Compatibilidad

¿Funciona bien junto a otros sistemas?

3.3.1 Co-existencia

El sistema convive con la infraestructura de red del cliente sin interferir:

- **No requiere apertura de puertos en el router del cliente.** La conectividad entre el nodo Edge local y la nube Oracle se establece mediante túneles salientes (*outbound tunnels*) de Cloudflare y ngrok. Esta decisión de diseño fue tomada explícitamente para maximizar la compatibilidad con redes corporativas y domésticas restrictivas.

3.3.2 Interoperabilidad

Sistema externo	Protocolo de integración	Estado
Cámaras IP (Sonoff, marca genérica)	RTSP estándar 554	✓ Integrado
MediaMTX (servidor de streaming)	API REST en puerto 9997	✓ Integrado
Telegram API	HTTPS polling api.telegram.org	✓ Integrado
MinIO (S3-compatible)	AWS S3 SDK estándar	✓ Integrado
Cloudflare Tunnel	CLI cloudflared + HTTPS	✓ Integrado
ngrok	CLI + API REST	✓ Integrado

Protocolo de streaming adoptado: Se eligió **HLS (HTTP Live Streaming)** en lugar de WebRTC por su compatibilidad universal con navegadores modernos y su funcionamiento sobre TCP, que es soportado por los túneles gratuitos utilizados.

Calificación: ★ ★ ★ ★ ★ (5/5)

3.4 Usabilidad

¿Es fácil de usar y entender?

3.4.1 Reconocibilidad

La interfaz web presenta una arquitectura de información clara:

- **Dashboard:** Vista principal con mosaico de cámaras en vivo y panel de alertas en tiempo real.
- **Configuración del Sistema:** Formularios etiquetados para modificar credenciales RTSP por cámara.
- **Gestión de Usuarios:** Tabla con acciones de crear, editar y eliminar usuarios.
- **Grabaciones:** Listado con reproductor integrado.

3.4.2 Aprendizaje

El sistema implementa el principio de "mínima sorpresa" para los usuarios finales:

- Los formularios muestran el texto "(dejar vacío para no cambiar)" en el campo de contraseña, evitando que el operador borre accidentalmente una contraseña existente.
- Los botones de acción se deshabilitan (**disabled**) hasta que el usuario haya modificado un campo (**dirty: true**), previniendo guardados accidentales.
- El indicador de estado "Analizando..." informa al usuario que el sistema está procesando, evitando la percepción de que el sistema se "colgó" durante el análisis LLaVA.

3.4.3 Protección contra Errores del Usuario

Protección	Implementación
No puedes eliminar tu propio usuario	Validación en DELETE /usuarios/{id}
No puedes cambiar tu propio rol	Validación en PUT /usuarios/{id}

Protección	Implementación
Botón de guardar deshabilitado si no hay cambios	Estado dirty en React
Confirmación antes de borrar alertas masivamente	Diálogo de confirmación en frontend

Calificación: ★ ★ ★ ★ (4/5)

3.5 Fiabilidad

¿Falla poco? ¿Se recupera solo?

3.5.1 Madurez

El sistema opera en producción en Oracle Cloud con una disponibilidad continua de 24/7. Los contenedores están configurados con **restart: unless-stopped**, garantizando que ante cualquier fallo del proceso, Docker reinicia el contenedor automáticamente.

3.5.2 Disponibilidad

```
# docker-compose.yml – Configuración de reinicio automático
postgres:
  restart: unless-stopped
redis:
  restart: unless-stopped
backend:
  restart: unless-stopped
telegram:
  restart: unless-stopped
```

Cada servicio tiene un **healthcheck** configurado que verifica su estado periódicamente:

```
# PostgreSQL
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  timeout: 5s
  retries: 5

# Redis
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 3s
  retries: 5
```

El backend no arranca hasta que PostgreSQL, Redis y MinIO hayan superado sus healthchecks (**condition: service_healthy**).

3.5.3 Tolerancia a Fallos

El sistema implementa múltiples mecanismos de tolerancia a fallos:

- **Worker Telegram con backoff exponencial:** Si el token no está configurado, el worker no colapsa el sistema; reintenta con intervalos crecientes e informa el error en el log.
- **Heartbeat system (salud.py):** Cada componente reporta su estado a la tabla **salud_servicios** cada 30 segundos. Si la base de datos no está disponible, el heartbeat falla silenciosamente sin detener el componente.
- **Modo degradado:** Si Redis cae, el sistema continúa operando. El heartbeat reporta **estado = "degradado"** para ese servicio.
- **IP de respaldo para cámaras:** Si la IP de una cámara no se encuentra en la tabla ARP, el sistema usa la IP de respaldo almacenada en la base de datos en lugar de detener el detector.

3.5.4 Recuperabilidad

- Los datos de alertas y grabaciones se almacenan en PostgreSQL con volúmenes persistentes (**postgres-data**).
- Los snapshots se almacenan en MinIO con volumen persistente (**minio-data**).
- Ante un reinicio total del servidor Oracle, todos los datos persisten y los contenedores se reinician automáticamente.

Calificación: ★ ★ ★ ★ (4/5)

3.6 Seguridad

¿Protege los datos y el acceso?

3.6.1 Confidencialidad

Dato sensible	Mecanismo de protección
Contraseñas de usuarios	Hasheadas con bcrypt (12 rondas) · Nunca almacenadas en texto plano
Token JWT	Firmado con HS256 · Clave secreta inyectada vía variable de entorno
Contraseñas RTSP de cámaras	No expuestas en el GET /camaras · Solo modificables vía PATCH autenticado
Token de Telegram	Almacenado en configuracion_sistema · Enmascarado en respuestas de API
Credenciales de base de datos	Inyectadas vía variables de entorno · Nunca hardcodeadas en el código

3.6.2 Integridad

```
# auth.py – Verificación de integridad del password con bcrypt
def verify_password(password: str, password_hash: str) -> bool:
    return _bcrypt.checkpw(
        _to_bytes_72(password),
        password_hash.encode("utf-8")
    )
```

Todas las contraseñas se almacenan como hash bcrypt `$2b$12$...`, garantizando que incluso con acceso directo a la base de datos no se pueda obtener la contraseña original.

3.6.3 No Repudio

Cada alerta generada por el sistema registra:

- Timestamp de detección
- Nombre de la cámara origen
- Frame capturado (snapshot en MinIO)
- Descripción semántica generada por LLaVA
- Estado de análisis y tiempo de procesamiento

Esto proporciona una **cadena de evidencia auditable** para cada incidente registrado.

3.6.4 Control de Acceso (RBAC)

El sistema implementa un **Control de Acceso Basado en Roles** con 3 niveles:

Rol	Permisos
Admin	Gestión completa de usuarios · Eliminar alertas · Ver configuración · Todas las acciones de Operador
Operador	Ver cámaras · Cambiar credenciales RTSP · Recargar cámaras · Ver alertas y grabaciones
Visor	Solo lectura: ver cámaras, alertas y grabaciones

```
# auth.py – Sistema de dependencias de FastAPI para control de acceso
def require_admin(user: dict = Depends(get_current_user)) -> dict:
    if user.get("rol") != "admin":
        raise HTTPException(403, "Requiere rol admin")
    return user

def require_operator(user: dict = Depends(get_current_user)) -> dict:
    if user.get("rol") not in ("admin", "operador"):
        raise HTTPException(403, "Requiere rol operador o admin")
    return user
```

[!NOTE] **Área de mejora identificada:** El CORS está configurado con `allow_origins=["*"]` para facilitar el desarrollo. En un ambiente de producción final se recomienda restringir al dominio del

frontend.

Calificación: ★ ★ ★ ★ ★ (5/5)

3.7 🛠️ Mantenibilidad

¿Es fácil de modificar, corregir y entender?

3.7.1 Modularidad

La arquitectura del sistema está dividida en módulos completamente independientes, cada uno con responsabilidad única:

```
codigo_fuente/
├── backend/
│   ├── api.py           # Capa de presentación (endpoints FastAPI)
│   ├── auth.py          # Autenticación y autorización (JWT + bcrypt)
│   ├── detector.py      # Lógica de detección (YOLO + captura RTSP)
│   ├── analizador.py    # Análisis semántico v1 (LLaVA)
│   ├── analizador2.py   # Análisis semántico v2 (LLaVA mejorado)
│   └── salud.py         # Monitoreo de heartbeats
├── base_datos/
│   └── db.py            # Capa de acceso a datos (pool PostgreSQL)
├── frontend/
│   └── src/             # Componentes React
├── telegram/
│   └── worker.py        # Notificaciones push (independiente)
└── tablas_base_de_datos/
    ├── schema.sql       # Esquema inicial
    └── migrations/      # Migraciones numeradas (001, 002, ...)
```

Cada módulo puede ser desarrollado, probado y desplegado de forma independiente.

3.7.2 Reusabilidad

- El módulo `db.py` expone una interfaz única `db` que es importada por todos los módulos que necesitan acceso a la base de datos, evitando duplicación de lógica de conexión.
- El sistema de `Heartbeat` en `salud.py` es una clase reutilizable que cualquier componente puede instanciar con una sola línea.
- Las funciones `require_admin` y `require_operator` son decoradores reutilizables de FastAPI que protegen cualquier endpoint con una sola instrucción.

3.7.3 Analizabilidad

Todos los módulos implementan logging estructurado con prefijos identificables:

```
log = logging.getLogger("camaras.auth")
log.info(f"[db] Conectado a {DB_NAME}@{DB_HOST}:{DB_PORT}")
```



```
log.error(f"[auth] Fallo verificación de contraseña: {e}")
```

Los logs del sistema son accesibles en tiempo real mediante `docker compose logs -f <servicio>`.

3.7.4 Capacidad de Modificación

La gestión de cambios en la base de datos se implementa mediante un **sistema de migraciones numeradas** en la carpeta `tablas_base_de_datos/migrations/`. Cada migración tiene un número de orden y es aplicada de forma incremental, garantizando que el esquema puede evolucionar sin perder datos.

Calificación: ★ ★ ★ ★ ★ (5/5)

3.8 Portabilidad

¿Se puede desplegar en otro entorno?

3.8.1 Adaptabilidad

El sistema fue diseñado desde el inicio para ser independiente del entorno de ejecución:

- **Toda la infraestructura está contenerizada** con Docker Compose. El stack completo de 6 contenedores se puede levantar en cualquier sistema Linux con Docker instalado mediante un solo comando: `docker compose up -d`.
- Las **variables de entorno** en `.env` separan la configuración del código. Migrar el sistema a otro proveedor de nube (AWS, GCP, Azure) requiere solo cambiar las IPs y credenciales en el archivo `.env`.
- El **esquema de base de datos** se aplica automáticamente en el primer arranque mediante el archivo `schema.sql` montado como volumen en el contenedor de PostgreSQL.

3.8.2 Instalabilidad

```
# Despliegue completo en un servidor nuevo (3 comandos)
git clone <repo>
cp .env.example .env      # Configurar variables
docker compose up -d      # Levanta los 6 contenedores
```

La imagen base de `postgres:16-alpine` y `redis:7-alpine` son imágenes ligeras y ampliamente disponibles en cualquier registro de contenedores.

3.8.3 Reemplazabilidad

Cada componente puede ser reemplazado de forma independiente:

Componente actual	Reemplazo posible	Impacto en el sistema
PostgreSQL 16	Cualquier DB compatible con psycopg3	Solo cambiar string de conexión
Redis 7	KeyDB, Valkey (compatibles con Redis)	Zero impacto (mismo protocolo)

Componente actual	Reemplazo posible	Impacto en el sistema
MinIO	AWS S3, Cloudflare R2	Solo cambiar endpoint y credenciales
LLaVA	GPT-4V, Gemini Vision	Solo reescribir <code>analizador.py</code>

Calificación: ★★★★★ (5/5)

4. Matriz de Trazabilidad — Pruebas vs. Características ISO 25010

Caso de Prueba	Caract. ISO 25010 evaluada
CP-AUTH-01 al CP-AUTH-06	Seguridad · Adecuación Funcional
CP-API-01 al CP-API-07	Adecuación Funcional · Eficiencia de Desempeño
CP-USR-01 al CP-USR-03	Seguridad · Usabilidad
CP-IA-01 al CP-IA-05	Adecuación Funcional · Eficiencia de Desempeño · Fiabilidad
CP-TEL-01 al CP-TEL-03	Fiabilidad · Adecuación Funcional
CP-INF-01 al CP-INF-03	Fiabilidad · Portabilidad · Compatibilidad

5. Conclusión de la Evaluación

El sistema Cámaras-IA demuestra un **nivel de calidad alto y sostenible** bajo el estándar ISO/IEC 25010. Las fortalezas principales del sistema son su **Seguridad** (RBAC granular, bcrypt, JWT), su **Mantenibilidad** (arquitectura modular y documentada) y su **Portabilidad** (contenerización Docker completa).

La única característica que presenta una restricción es la **Eficiencia de Desempeño**, específicamente en la latencia del análisis semántico de LLaVA en CPU (~120 segundos). Esta restricción es una consecuencia deliberada del uso de infraestructura gratuita y está debidamente documentada con una ruta de escalamiento clara hacia instancias con GPU.

5.1 Plan de Mejora Continua

Prioridad	Mejora	Característica ISO 25010
Alta	Restringir CORS a dominio del frontend en producción	Seguridad
Alta	Migrar a instancia con GPU para reducir latencia LLaVA	Eficiencia de Desempeño
Media	Implementar suite de pruebas automatizadas con pytest	Mantenibilidad
Media	Agregar autenticación con 2FA (TOTP)	Seguridad
Baja	Implementar versionado de la API (<code>/v1/...</code>)	Compatibilidad

Evaluación realizada conforme a ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation Proyecto de Título Cámaras-IA · Duoc UC · Junio 2026